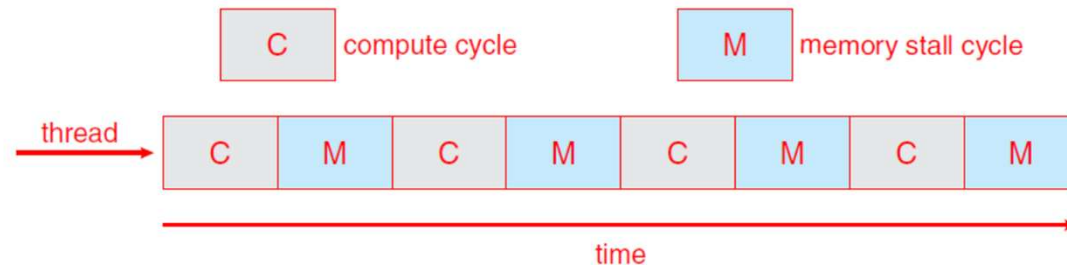
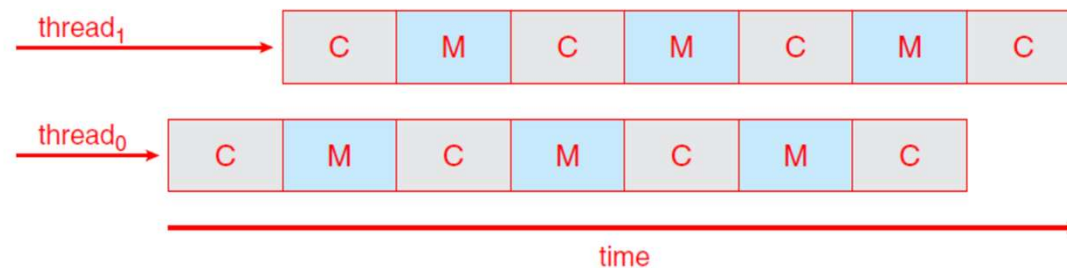


Scheduling Multi-Processore

- ❑ Sistemi multicore
- ❑ Ogni core appare al SO come una CPU separata
- ❑ Core veloci, ma problemi di **memory stall** attesa di dati pronti in memoria

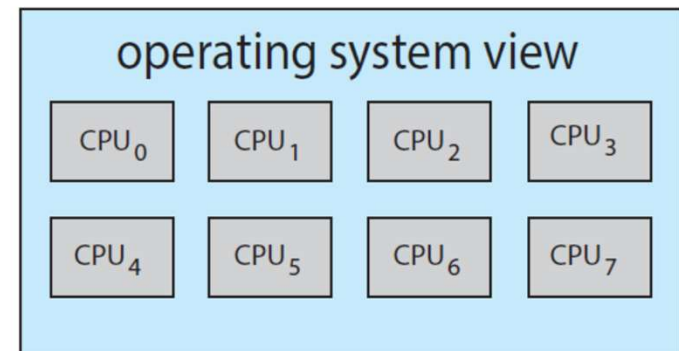
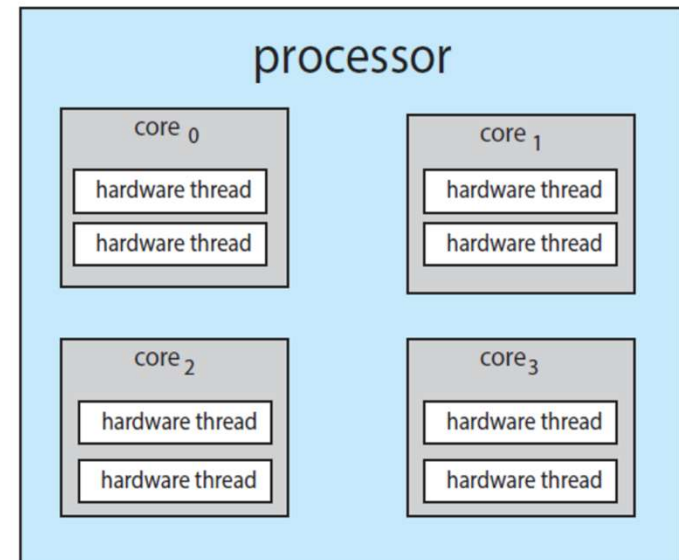


- ❑ Più thread pronti per core: **chip multithreading (CMT)**



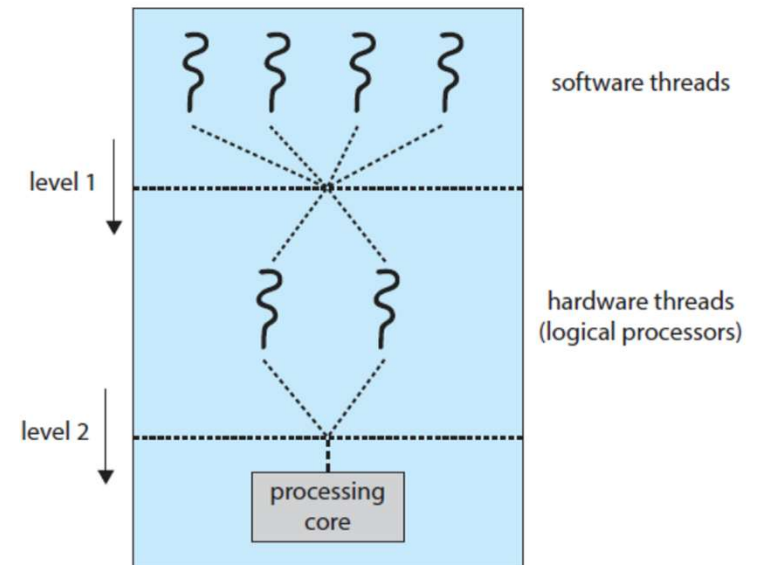
Scheduling Multi-Processore

- ❑ Sistemi multicore
- ❑ Ogni core appare al SO come una CPU separata
- ❑ Core veloci, ma problemi di **memory stall** attesa di dati pronti in memoria
- ❑ Più thread pronti per core: **chip multithreading (CMT)**
- ❑ Per SO è come avere più CPU
intel chiama hyperthreading
- ❑ Es. Oracle Spark M7 8 thread per core su 8 core, come 64 CPU



Scheduling Multi-Processore

- ❑ Sistemi multicore
- ❑ Ogni core appare al SO come una CPU separata
- ❑ Core veloci, ma problemi di **memory stall** attesa di dati pronti in memoria
- ❑ Più thread pronti per core: **chip multithreading (CMT)**
- ❑ Per OS è come avere più CPU
intel chiama hyperthreading
- ❑ Es. Oracle Spark M7 8 thread per core su 8 core, come 64 CPU
- ❑ Due livelli di scheduling:
 - ❑ Software threads (SO)
 - ❑ Hardware threads

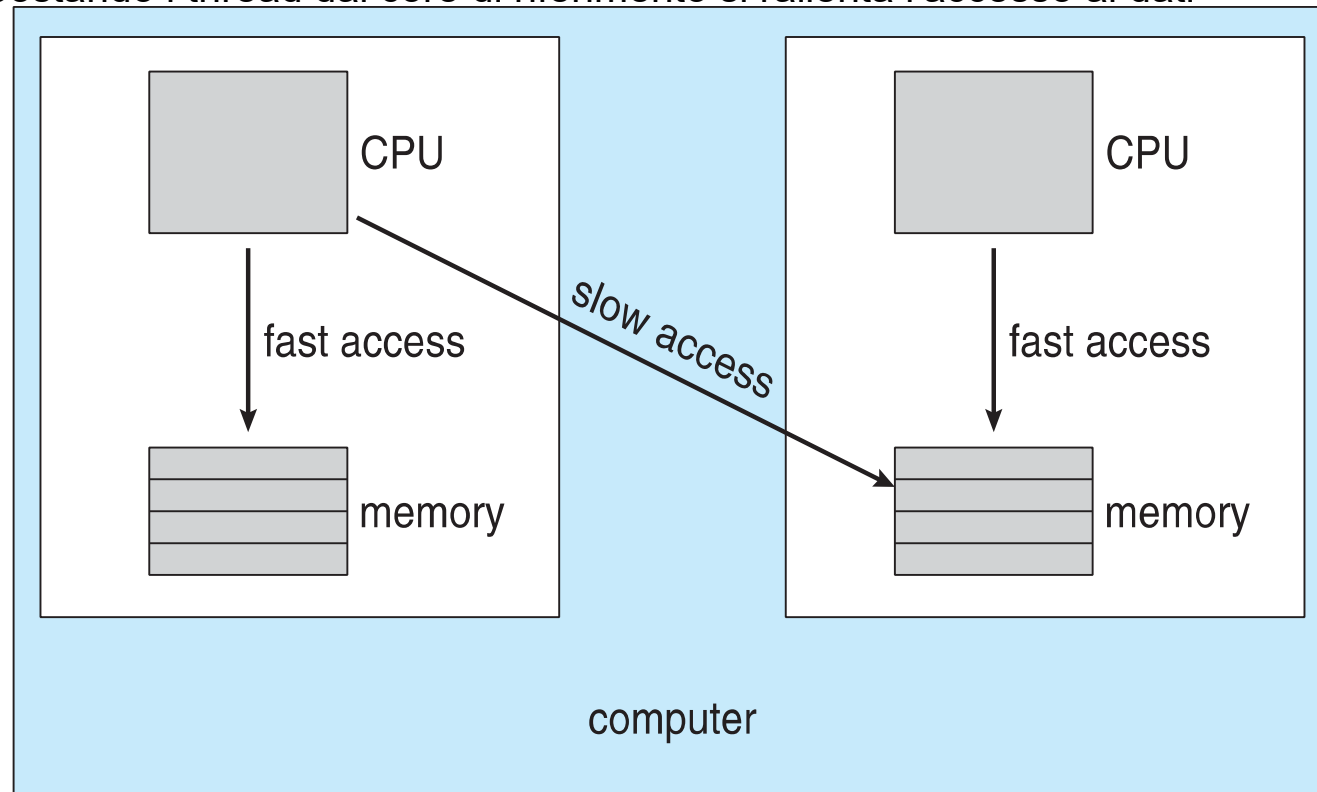


Multiple-Processor Scheduling – Load Balancing

- Se Symmetric Multiprocessing (SMP) e code private:
 - **Load balancing**: necessario bilanciare il carico tra i processori
- Due metodi di Load balancing:
 - **Push migration** – un processo verifica continuamente il carico dei processori, se trova uno sbilanciamento sposta/spinge il task dalla CPU sovraccarica alle altre
 - **Pull migration** – i processori in idle prendono/tirano i task in attesa sulle code dei processori occupati
 - Esempio: Linux implementa entrambe le strategie
- **Processor affinity** – processo ha affinità per il processore su cui è in running
 - **soft affinity** – tentativo di mantenere il processore del thread
 - **hard affinity** – specifica il sottoinsieme di processori per il thread
 - Esempio, Linux implementa soft, ma supporta entrambi
 - Con code private per core più semplice affinity

NUMA e CPU Scheduling

- Non-uniform Memory Access (NUMA)
 - Due chip con la propria CPU e memoria
 - Accessi in memoria a velocità diversa
 - C'è un conflitto tra problematiche di accesso in memoria e load balancing
 - ▶ Spostando i thread dai core di riferimento si rallenta l'accesso ai dati



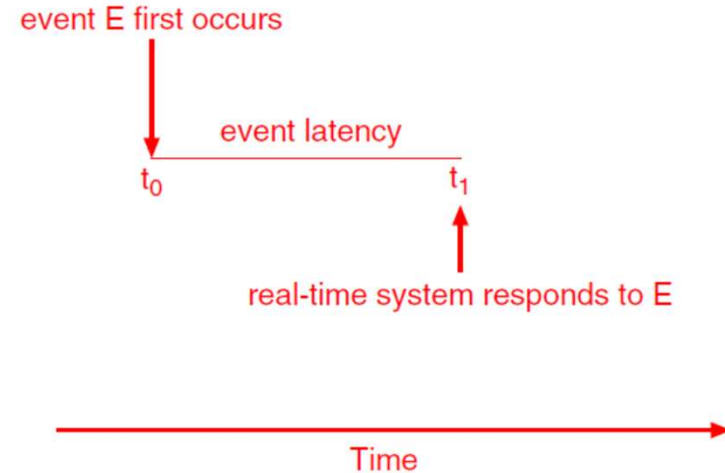
Gli algoritmi di memory-placement possono considerare l'affinity

Heterogeneous Multiprocessing

- Alcuni sistemi hanno architetture multicore non omogenee
 - Diverso clock, diverso consumo di energia, etc.
 - Non asimmetrici (perché core omogenei per capacità di calcolo)
 - ma diversi consumi: Big core e Little core (processori ARM)
 - ▶ Big core veloci ma consumano
 - ▶ Little core lenti ma economici
 - ▶ Vantaggio per sistemi mobile
 - ▶ **Algoritmi di scheduling finalizzati al risparmio di energia**

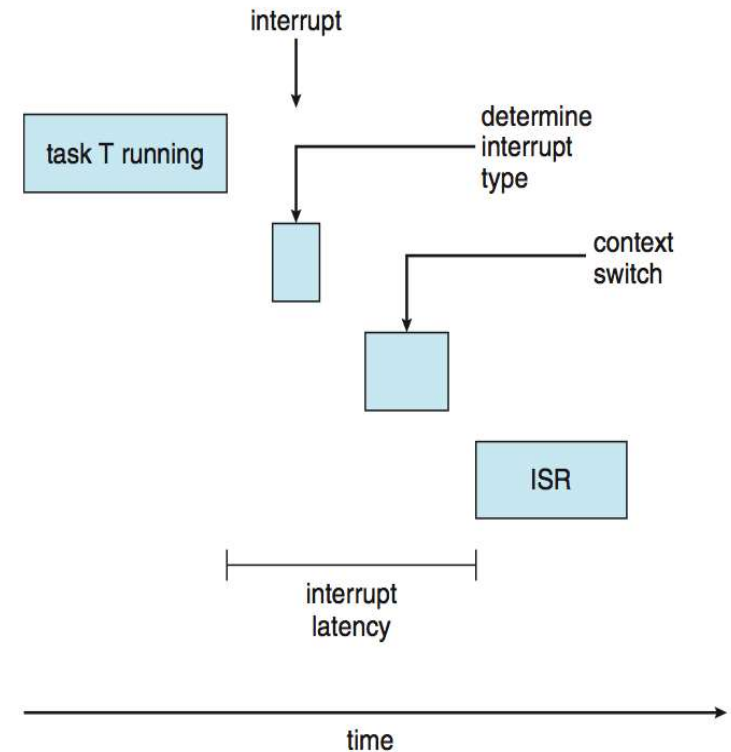
Real-Time CPU Scheduling

- Sistemi Real-Time
- **Soft real-time system**
 - non garanzie su quando un processo verrà schedulato, solo processi critici preferiti a non critici
- **Hard real-time system**
 - task servito prima di una deadline
- Latenza di un evento:
 - Tempo trascorso tra evento e risposta del sistema



Real-Time CPU Scheduling

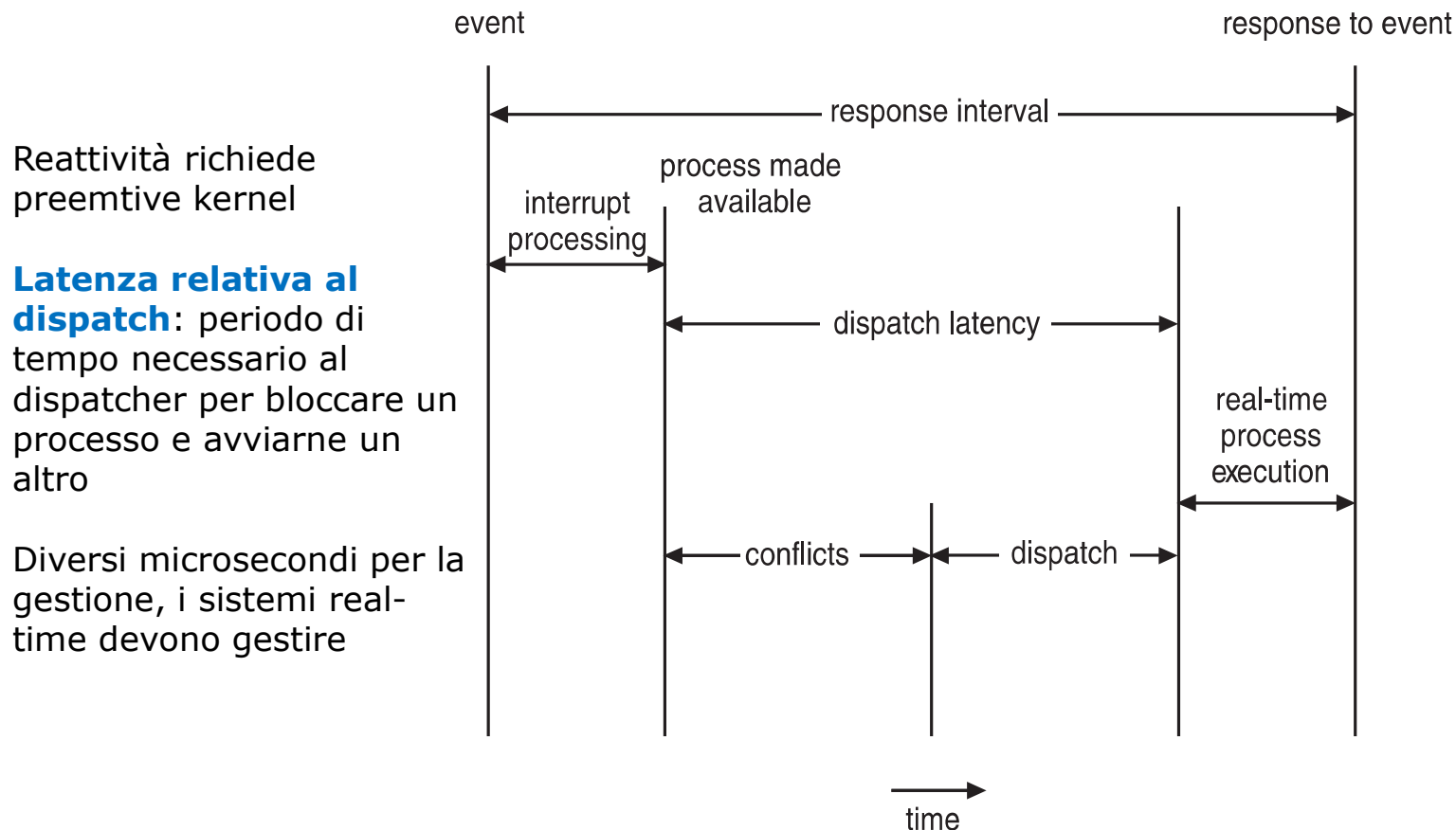
- Sistemi Real-Time
- **Soft real-time system**
 - non garanzie su quando un processo verrà schedulato, solo processi critici preferiti a non critici
- **Hard real-time system**
 - task servito prima di una deadline
- Due tipi di latenze incidono sulla prestazione
 1. **Interrupt latency** – tempo dall'arrivo dell'interrupt allo start della routine di servizio
 2. **Dispatch latency** – tempo di scheduling per fare lo switch di un altro processo



Durante gli update delle strutture del kernel gli interrupt sono disabilitati, in sistemi real time, va minimizzato questo tempo

Real-Time CPU Scheduling

- **Dispatch latency**: fase di conflitto e fase di dispatch del processo
- Fase di gestione del **conflitto** durante la dispatch latency:
 1. Prelazione di qualunque processo in kernel mode
 2. Rilascio delle risorse occupate dai processi a bassa priorità quando richieste dai processi ad alta priorità

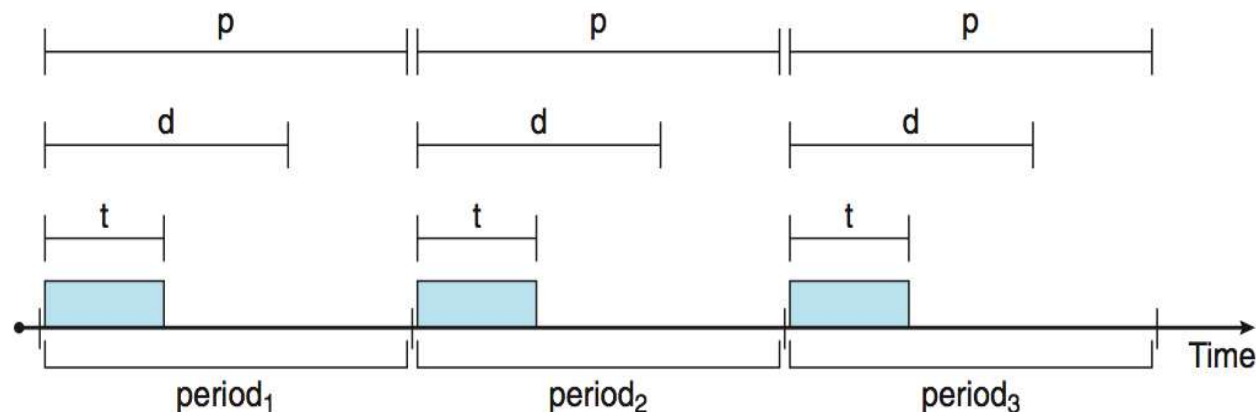


Priority-based Scheduling

- Nel real-time scheduling lo scheduler deve supportare prelazione e priority-based scheduling
 - Es. Windows ha 32 livelli di priorità con i livelli da 16 a 31 per i processi real-time
 - Ma questo garantisce solo il soft real-time
- Per hard real-time deve anche garantire le deadlines
 - Servono altri meccanismi ed altre politiche di schedulazione

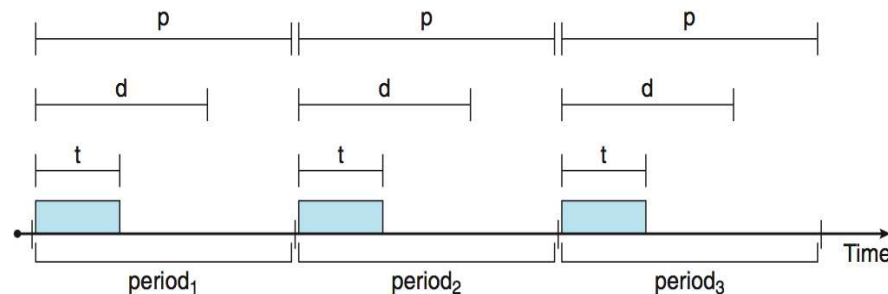
Priority-based Scheduling

- Nel real-time scheduling lo scheduler deve supportare preemptive e priority-based scheduling
 - Ma questo garantisce solo il soft real-time
- Per hard real-time deve anche garantire le deadlines
- Consideriamo task **periodici**
 - Richiedono la CPU a intervalli costanti
 - Ottenuta la CPU ha un tempo di processo t , deadline d , e periodo p
 - $0 \leq t \leq d \leq p$
 - **Rate** di un task periodico $1/p$



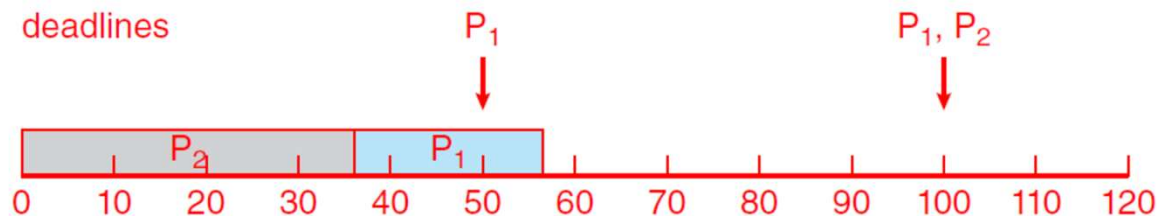
Priority-based Scheduling

- Nel real-time scheduling lo scheduler deve supportare preemptive e priority-based scheduling
 - Ma questo garantisce solo il soft real-time
- Per hard real-time deve anche garantire le deadlines
- Consideriamo task **periodici**
 - Richiedono la CPU a intervalli costanti
 - Ottenuta la CPU ha un tempo di processo t , deadline d , e periodo p
 - $0 \leq t \leq d \leq p$
 - **Rate** di un task periodico $1/p$
 - **Admission control algorithm**: il processo dichiara una deadline, lo scheduler può ammettere il processo garantendo il rispetto della deadline oppure non ammettere



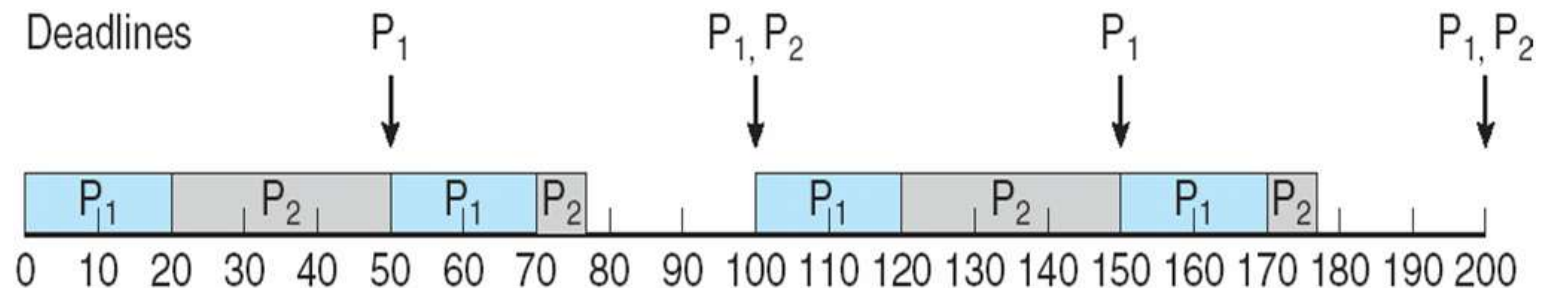
Rate Monotonic Scheduling

- Priorità assegnata in base all'inverso del periodo
 - Periodi brevi = priorità alta;
 - Periodi lunghi = priorità bassa
 - Priorità alta a chi interviene più spesso
 - Ogni processo mantiene lo stesso CPU burst
- Esempio:
 - Periodi 50 e 100, tempo di processamento rispettivamente 20 e 35
 - Deadline: terminare prima del prossimo periodo
 - Uso CPU per P_1 di è $20/50 = 0.4$, per P_2 di è $35/100 = 0.35$, tot 0.75
 - Assumendo P_2 con priorità più alta di P_1 deadline persa per P_1



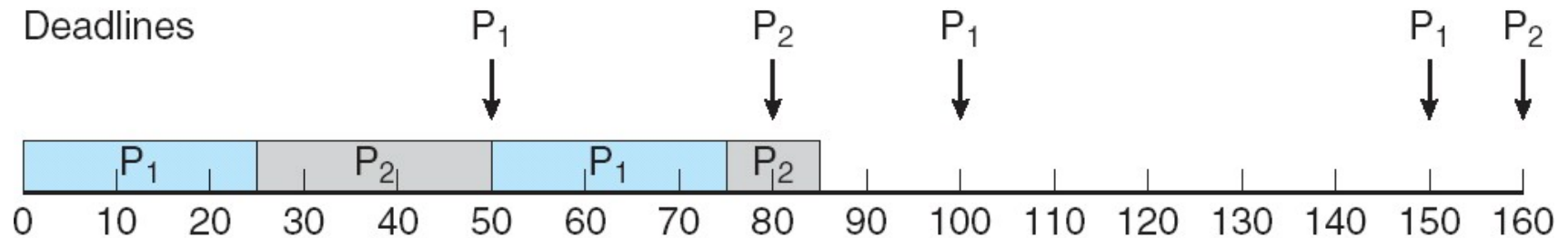
Rate Monotonic Scheduling

- Priorità assegnata in base all'inverso del periodo
- Periodi brevi = priorità alta;
- Periodi lunghi = priorità bassa
- Esempio:
 - Periodi 50 e 100, tempo di processamento 20, 35
 - Deadline: terminare prima del prossimo period
 - Uso CPU per P_1 di è $20/50 = 0.4$, per P_2 di è $35/100 = 0.35$, tot 0.75
 - Assumendo RMS:
 - ▶ P_1 con priorità più alta di P_2 entrambe le deadline rispettate



Deadline persa con il Rate Monotonic Scheduling

- Consideriamo un caso in cui non si riesce a schedulare
 - Se P_1 ha periodo 50 e CPU brust 25 e P_2 periodo 80 e CPU brust 35
 - Utilizzo CPU $25/50 + 35/80 = 0.94$
 - P_2 interrotta da P_1 quando riprende finisce a 85, ma doveva finire ad 80



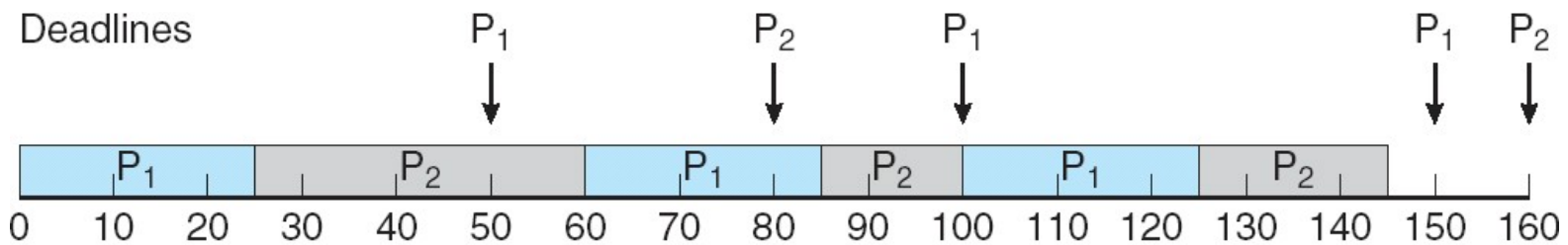
- Non sfrutta completamente la CPU, l'utilizzo diminuisce con il numero di processi

$$N(2^{1/N} - 1)$$

- Con 1 processo 100%, con 2 processi 83%, con al crescere di N 69%

Earliest Deadline First Scheduling (EDF)

- Priorità sono assegnate secondo le deadline:
 - prima la deadline, più alta è la priorità; più lontana è la deadline, più bassa è la priorità (le priorità non sono fisse, variano nel tempo)



P1: $p_1 = 50$, $t_1 = 25$; P2: $p_2 = 80$, $t_2 = 35$

Non richiede la periodicità dei processi e neppure CPU burst costante

Ogni processo deve dichiarare la deadline quando diventa runnable

Teoricamente ottimo (se utilizzo sotto 100% di CPU rispetta la deadline) però va considerate il context switch e il costo della gestione dell'interrupt

Earliest Deadline First Scheduling (EDF)

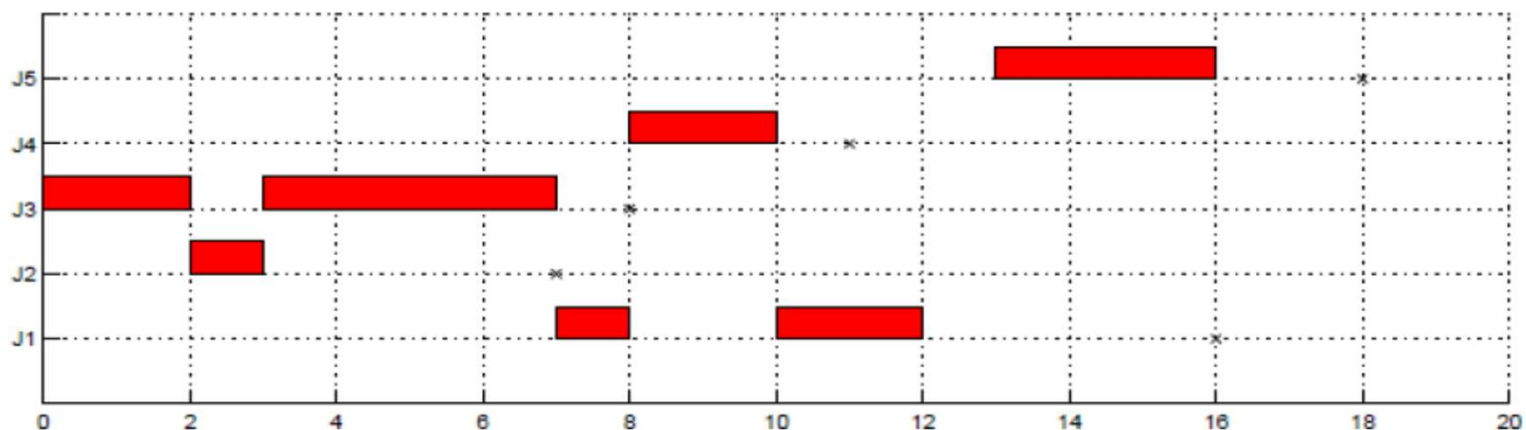
- Priorità sono assegnate secondo le deadline
- Esempio:
 - Tracciare il Gantt secondo EDF verificando se le scadenze sono rispettate (scheduling ammissibile)

Processo	Tempo di arrivo	Esecuzione	Scadenza
J_1	0	3	16
J_2	2	1	7
J_3	0	6	8
J_4	8	2	11
J_5	13	3	18

Earliest Deadline First Scheduling (EDF)

- Priorità sono assegnate secondo le deadline
- Esempio:
 - Tracciare il Gantt secondo EDF verificando se le scadenze sono rispettate (ammissibile)

Processo	Tempo di arrivo	Esecuzione	Scadenza
J_1	0	3	16
J_2	2	1	7
J_3	0	6	8
J_4	8	2	11
J_5	13	3	18



Earliest Deadline First Scheduling (EDF)

- Priorità sono assegnate secondo le deadline:
 - prima la deadline, più alta è la priorità; più lontana è la deadline, più bassa è la priorità (le priorità non sono fisse, variano nel tempo)

Non richiede la periodicità dei processi e neppure CPU burst costante

Ogni processo deve dichiarare la deadline quando diventa runnable

Teoricamente ottimo (se utilizzo sotto 100% di CPU rispetta la deadline) però va considerate il context switch e il costo della gestione dell'interrupt

Se un insieme di processi caratterizzati da: tempo di arrivo, un requisito di esecuzione, scadenza può essere schedulata (da un algoritmo) in modo da garantire tutte le scadenze, l'EDF garantirà la scadenza.

Earliest Deadline First Scheduling (EDF)

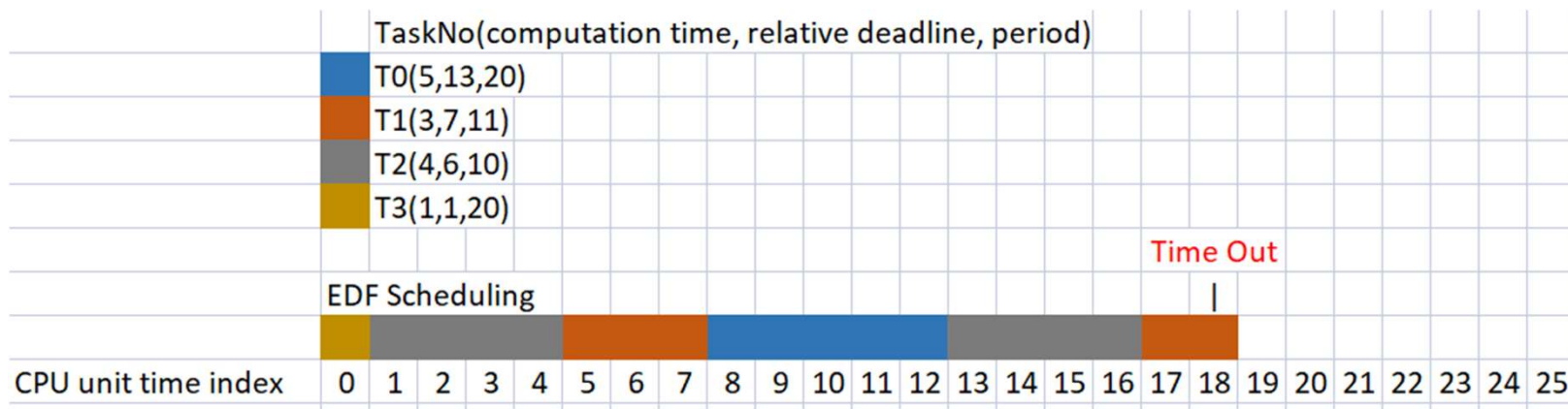
- Priorità sono assegnate secondo le deadline:
 - prima la deadline, più alta è la priorità; più lontana è la deadline, più bassa è la priorità (le priorità non sono fisse, variano nel tempo)

Attenzione si è assunto $\text{deadline} = \text{periodo}$.

Se la deadline è prevista prima del periodo non garantisce.

Es. $T_0(5,13,20)$, $T_1(3,7,11)$, $T_2(4,6,10)$, $T_3(1,1,20)$

Utilizzo $5/20 + 3/11 + 4/10 + 1/20 = 0.97$



Scheduling a Quote Proporzionali

- Quote proporzionali di CPU preallocate per le applicazioni
- T quote di CPU devono essere preallocate tra tutti i processi
- Un'applicazione riceve N quote con $N < T$
 - L'algoritmo assicura l'applicazione riceverà N / T del tempo totale del processore
- Esempio $T = 100$ da dividere su A, B, C. A che riceve 50, B riceve 15, e C riceve 20. Rimangono quote per altri processi
- **Algoritmo di admission control** assegna le quote rimanenti ai nuovi processi nel rispetto delle quote già assegnate

POSIX Real-Time Scheduling

POSIX.1b fornisce API standard per gestire real-time threads

Definisce due classi di scheduling per real-time threads:

1. `SCHED_FIFO` - threads sono schedulati con strategia FCFS con coda FIFO. No time-slicing per thread con priorità uguale
2. `SCHED_RR` - simile a `SCHED_FIFO` ma con time-slicing per thread di pari priorità

Definisce due funzioni per prendere e settare la politica di scheduling:

1. `pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`
2. `pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)`

POSIX Real-Time Scheduling API

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[])
{
    int i, policy;
    pthread_t_tid[NUM_THREADS];
    pthread_attr_t attr;
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* get the current scheduling policy */
    if (pthread_attr_getschedpolicy(&attr, &policy) != 0)
        fprintf(stderr, "Unable to get policy.\n");
    else {
        if (policy == SCHED_OTHER) printf("SCHED_OTHER\n");
        else if (policy == SCHED_RR) printf("SCHED_RR\n");
        else if (policy == SCHED_FIFO) printf("SCHED_FIFO\n");
    }
}
```

POSIX Real-Time Scheduling API (Cont.)

```
/* set the scheduling policy - FIFO, RR, or OTHER */
if (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0)
    fprintf(stderr, "Unable to set policy.\n");
/* create the threads */
for (i = 0; i < NUM_THREADS; i++)
    pthread_create(&tid[i], &attr, runner, NULL);
/* now join on each thread */
for (i = 0; i < NUM_THREADS; i++)
    pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
    /* do some work ... */
    pthread_exit(0);
}
```


Hard real-time

Il POSIX 1b supportato a partire da kernel Linux (dal 2.6) fornendo schedulazione con priorità, supporto per segnali, sincronizzazione etc.

Per avere garanzie hard-real time occorre un kernel adatto:
Real-Time Operating System (RTOS)

Es. In Linux PREEMPT_RT Kernel (patch)

Altri approcci basati su cokernel (es., RTLinux, Xenomai, RTAI)

Virtualizzazione e Scheduling

- Con Virtualizzazione si schedula con SO ospiti multipli sulla CPU(s)
- Ogni ospite esegue il suo scheduling
 - Ma in realtà non ha una sua CPU e sta schedulando pensando di avere l'intera CPU
 - Può portare a risposte rallentate
 - Lo stesso clock del Sistema può essere rallentato
- Si possono perdere i benefici di un buon algoritmo di scheduling implementato sul SO ospite